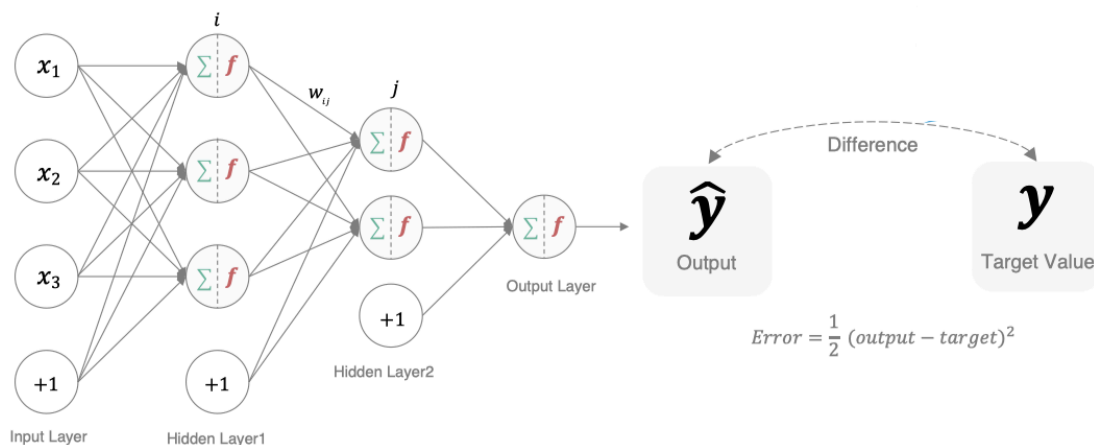


2.3 深度学习的优化方法

学习目标

- 知道梯度下降算法
- 理解神经网络的链式法则
- 掌握反向传播算法（BP算法）
- 知道梯度下降算法的优化方法
- 了解学习率退火

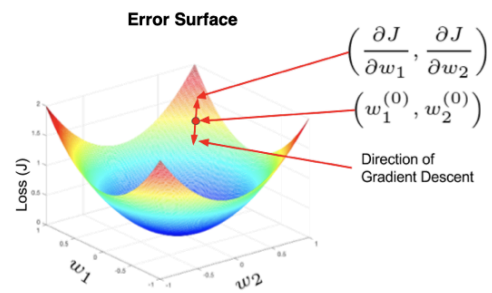
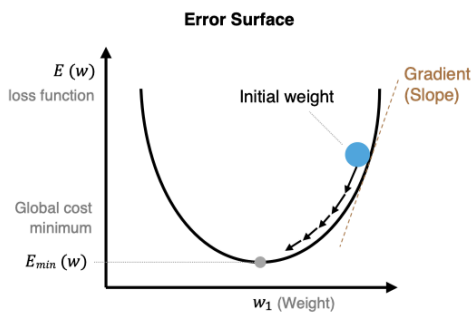


1. 梯度下降算法【回顾】

梯度下降法简单来说就是一种寻找使损失函数最小化的方法。大家在机器学习阶段已经学过该算法，所以我们在这里就简单的回顾下，从数学上的角度来看，梯度的方向是函数增长速度最快的方向，那么梯度的反方向就是函数减少最快的方向，所以有：

$$w_{ij}^{new} = w_{ij}^{old} - \eta \frac{\partial E}{\partial w_{ij}}$$

其中， η 是学习率，如果学习率太小，那么每次训练之后得到的效果都太小，增大训练的时间成本。如果，学习率太大，那就有可能直接跳过最优解，进入无限的训练中。解决的方法就是，学习率也需要随着训练的进行而变化。



在上图中我们展示了一维和多维的损失函数，损失函数呈碗状。在训练过程中损失函数对权重的偏导数就是损失函数在该位置点的梯度。我们可以看到，沿着负梯度方向移动，就可以到达损失函数底部，从而使损失函数最小化。这种利用损失函数的梯度迭代地寻找局部最小值的过程就是梯度下降的过程。

根据在进行迭代时使用的样本量，将梯度下降算法分为以下三类：

梯度下降算法	定义	缺点	优点
BGD（批量梯度下降）	每次迭代时需要计算每个样本上损失函数的梯度并求和	计算量大、迭代速度慢	全局最优化
SGD（随机梯度下降）	每次迭代时只采集一个样本，计算这个样本损失函数的梯度并更新参数	准确度下降、存在噪音、非全局最优化	训练速度快、支持在线学习
MBGD（小批量梯度下降）	每次迭代时，我们随机选取一小部分训练样本来计算梯度并更新参数	准确度不如BGD、非全局最优解	计算小批量数据的梯度更加高效、支持在线学习

实际中使用较多的是小批量的梯度下降算法，在tf.keras中通过以下方法实现：

```
tf.keras.optimizers.SGD(
    learning_rate=0.01, momentum=0.0, nesterov=False, name='SGD', **kwargs
)
```

例子：

```
# 导入相应的工具包
import tensorflow as tf
# 实例化优化方法：SGD
opt = tf.keras.optimizers.SGD(learning_rate=0.1)
# 定义要调整的参数
var = tf.Variable(1.0)
# 定义损失函数：无参但有返回值
loss = lambda: (var ** 2)/2.0
# 计算梯度，并对参数进行更新，步长为 `learning_rate * grad`
opt.minimize(loss, [var]).numpy()
# 展示参数更新结果
var.numpy()
```

更新结果为：

```
# 1-0.1*1=0.9
0.9
```

在进行模型训练时，有三个基础的概念：

名词	定义
Epoch	使用训练集的全部数据对模型进行一次完整训练，被称之为“一代训练”
Batch	使用训练集中的一小部分样本对模型权重进行一次反向传播的参数更新，这一小部分样本被称为“一批数据”
Iteration	使用一个 Batch 数据对模型进行一次参数更新的过程，被称之为“一次训练”

实际上，梯度下降的几种方式的根本区别就在于 Batch Size不同，如下表所示：

梯度下降方式	Training Set Size	Batch Size	Number of Batches
BGD	N	N	1
SGD	N	1	N
Mini-Batch	N	B	$N/B + 1$

注：上表中 Mini-Batch 的 Batch 个数为 $N/B + 1$ 是针对未整除的情况。整除则是 N/B 。

假设数据集有 50000 个训练样本，现在选择 Batch Size = 256 对模型进行训练。

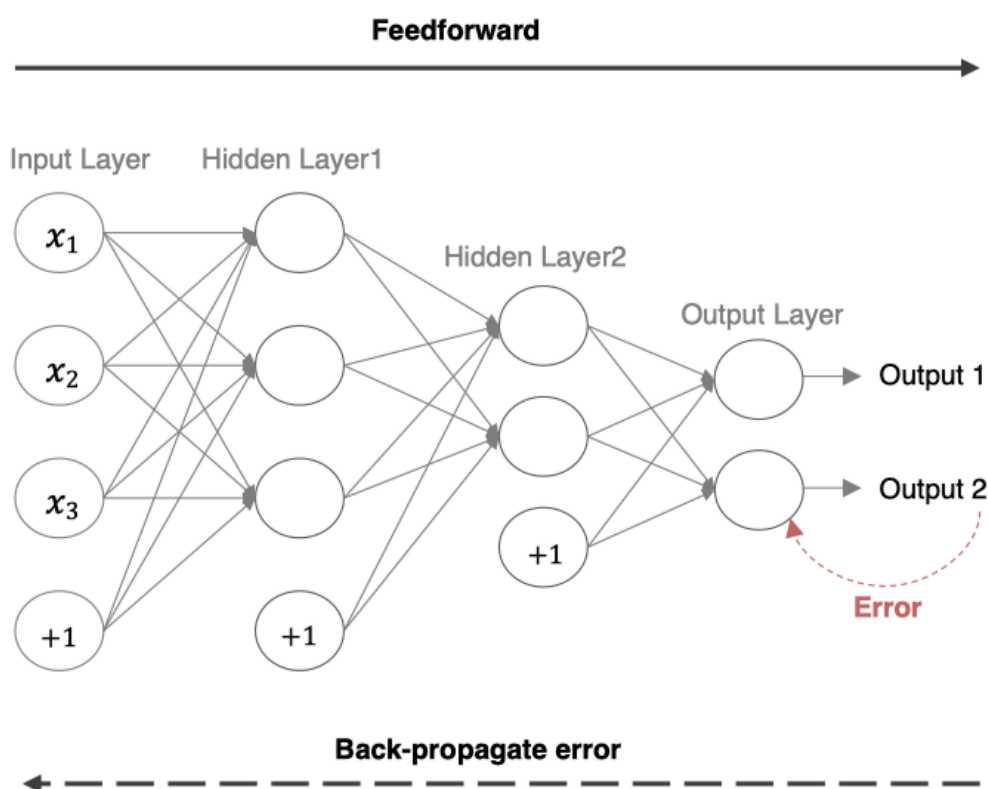
- 每个 Epoch 要训练的图片数量：50000
- 训练集具有的 Batch 个数： $50000/256+1=196$
- 每个 Epoch 具有的 Iteration 个数：196
- 10个 Epoch 具有的 Iteration 个数：1960

2.反向传播算法（BP算法）

利用反向传播算法对神经网络进行训练。该方法与梯度下降算法相结合，对网络中所有权重计算损失函数的梯度，并利用梯度值来更新权值以最小化损失函数。在介绍BP算法前，我们先看下前向传播与链式法则的内容。

2.1 前向传播与反向传播

前向传播指的是数据输入的神经网络中，逐层向前传输，一直到运算到输出层为止。



在网络的训练过程中经过前向传播后得到的最终结果跟训练样本的真实值总是存在一定误差，这个误差便是损失函数。想要减小这个误差，就用损失函数ERROR，从后往前，依次求各个参数的偏导，这就是反向传播（Back Propagation）。

2.2 链式法则

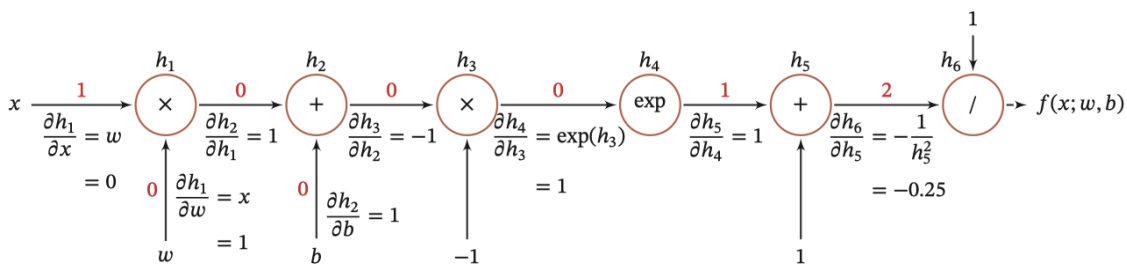
反向传播算法是利用链式法则进行梯度求解及权重更新的。对于复杂的复合函数，我们将其拆分为一系列的加减乘除或指数，对数，三角函数等初等函数，通过链式法则完成复合函数的求导。为简单起见，这里以一个神经网络中常见的复合函数的例子来说明这个过程。令复合函数 $f(x; w, b)$ 为：

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}$$

其中 x 是输入数据， w 是权重， b 是偏置。我们可以将该复合函数分解为：

函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

并进行图形化表示，如下所示：



整个复合函数 $f(x; w, b)$ 关于参数 w 和 b 的导数可以通过 $f(x; w, b)$ 与参数 w 和 b 之间路径上所有的导数连乘来得到，即：

$$\frac{\partial f(x; w, b)}{\partial w} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w},$$

$$\frac{\partial f(x; w, b)}{\partial b} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial b}.$$

以 w 为例，当 $x = 1, w = 0, b = 0$ 时，可以得到：

$$\begin{aligned}\frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25.\end{aligned}$$

注意：常用函数的导数：

函数	导数
$f(a) = a^2$	$2a$
$f(a) = a^3$	$3a^2$
$f(a) = \ln(a)$	$\frac{1}{a}$
$f(a) = e^a$	e^a
$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - (\tanh(z))^2 = 1 - (g(z))^2$

注：sigmoid的导数计算

$$\sigma(x) = \frac{1}{1 + e^{-x}} \rightarrow \frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

2.3 反向传播算法

反向传播算法利用链式法则对神经网络中的各个节点的权重进行更新。我们通过一个例子来给大家介绍整个流程。

输出层权重：

$$w_{jk} = w_{jk} - \eta \frac{\partial E}{\partial w_{jk}}$$

隐藏层权重：

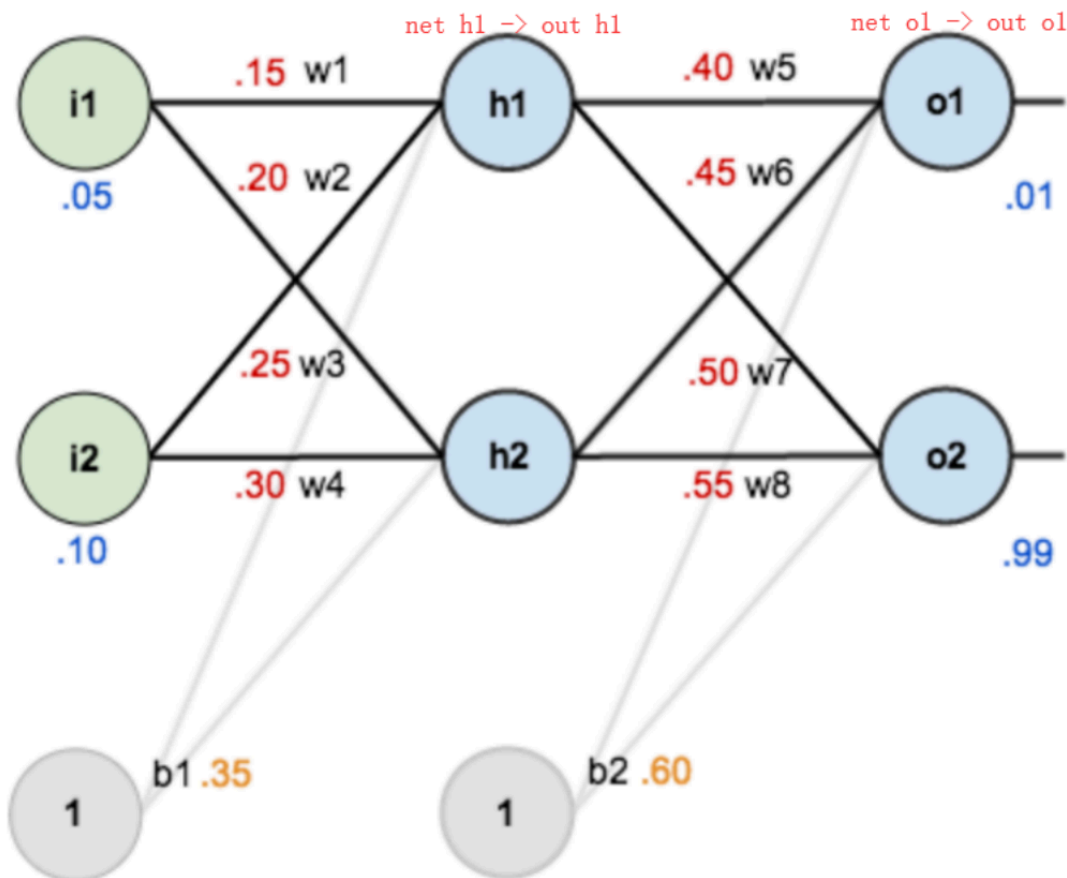
$$w_{ij} = w_{ij} - \eta \frac{\partial E}{\partial w_{ij}}$$

偏置更新：

$$b_j = b_j - \eta \frac{\partial E}{\partial b_j}$$

【举个栗子🌰：】

如下图是一个简单的神经网络用来举例：激活函数为sigmoid



前向传播运算：

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

$$net_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 = 0.3775$$

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

$$out_{h2} = 0.596884378$$



$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$net_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905967}} = 0.75136507$$

$$out_{o2} = 0.772928465$$

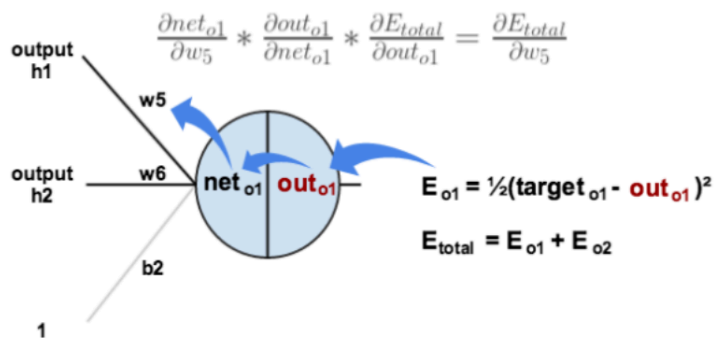


$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

接下来是**反向传播**（求网络误差对各个权重参数的梯度）：

我们先来求最简单的，求误差E对w5的导数。首先明确这是一个“**链式法则**”的求导过程，要求误差E对w5的导数，需要先求误差E对out o1的导数，再求out o1对net o1的导数，最后再求net o1对w5的导数，经过这个**链式法则**，我们就可以求出误差E对w5的导数（偏导），如下图所示：



$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

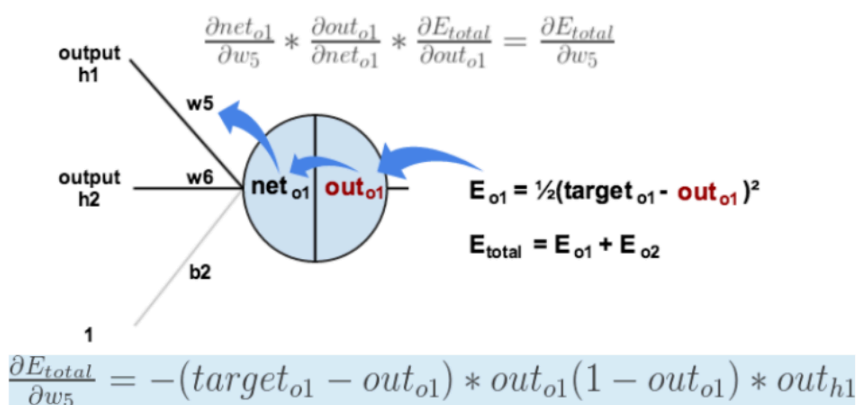
$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

导数（梯度）已经计算出来了，下面就是**反向传播与参数更新过程**：



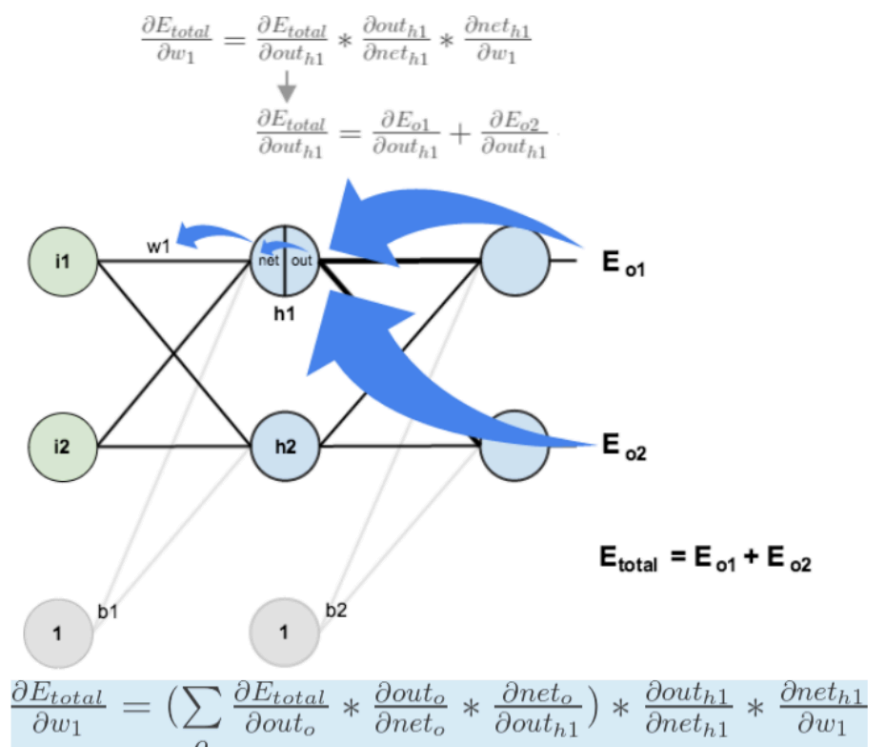
$$w_5^+ = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

如果要想求**误差E对w1的导数**，误差E对w1的求导路径不止一条，这会稍微复杂一点，但换汤不换药，计算过程如下所示：



$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

同理得到：

$$\begin{aligned} w_2^+ &= 0.19956143 \\ w_3^+ &= 0.24975114 \\ w_4^+ &= 0.29950229 \end{aligned}$$

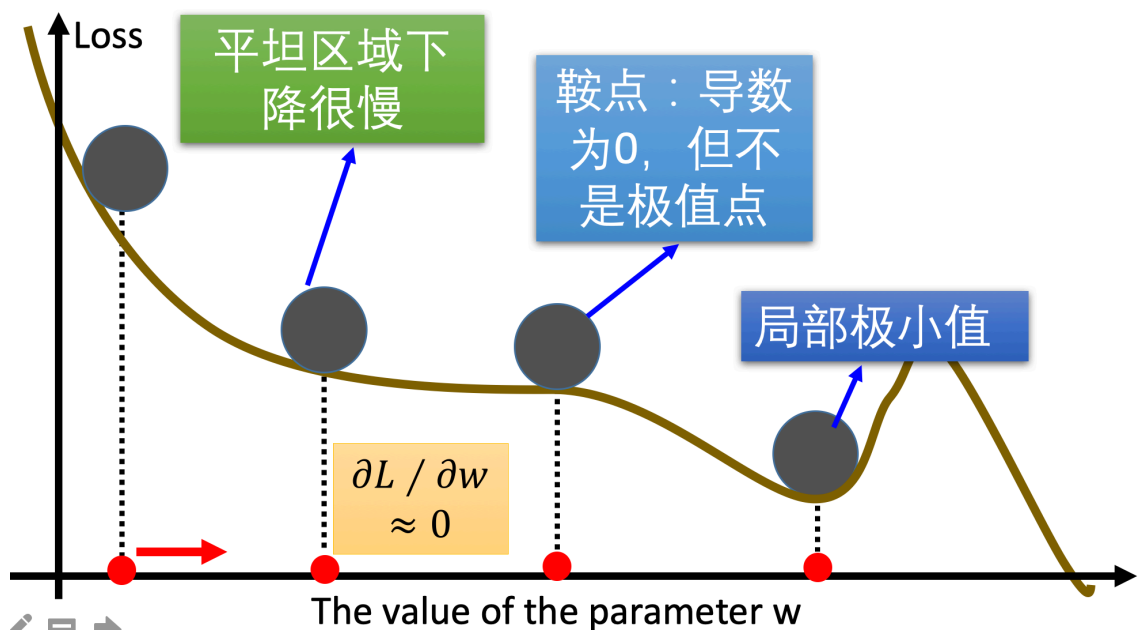
最后，更新了所有的权重！当最初前馈传播时输入为 0.05 和 0.1，网络上的误差是 0.298371109。在第一轮反向传播之后，总误差现在下降到 0.291027924。它可能看起来不多，但是在重复此过程 10000 次之后，例如：错误倾斜到 0.000035085。

在这一点上，当前馈输入为 0.05 和 0.1 时，两个输出神经元产生 0.015912196（相对于目标为 0.01）和 0.984065734（相对于目标为 0.99）

至此，**“反向传播算法”**的过程就讲完了啦！

3.梯度下降优化方法

梯度下降算法在进行网络训练时，会遇到鞍点，局部极小值这些问题，那我们怎么改进SGD呢？在这里我们介绍几个比较常用的

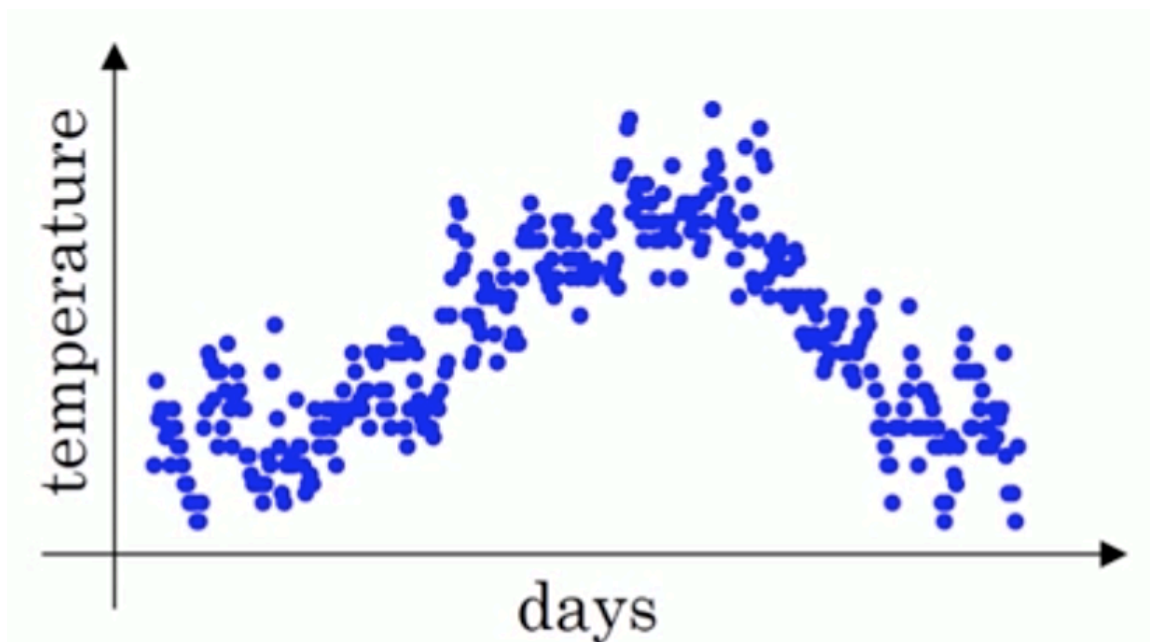


3.1 动量算法 (Momentum)

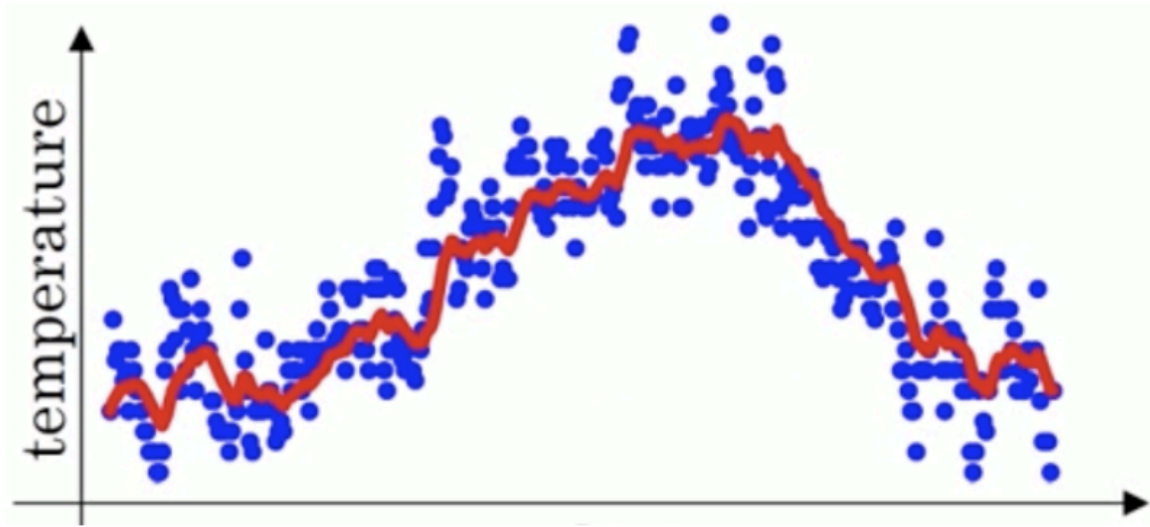
动量算法主要解决鞍点问题。在介绍动量法之前，我们先来看下指数加权平均数的计算方法。

指数加权平均

假设给定一个序列，例如北京一年每天的气温值，图中蓝色的点代表真实数据，



这时温度值波动比较大，那我们就使用加权平均值来进行平滑，如下图红线就是平滑后的结果：



计算方法如下所示：

$$S_t = \begin{cases} Y_1, & t = 1 \\ \beta S_{t-1} + (1 - \beta)Y_t, & t > 1 \end{cases}$$

其中 Y_t 为 t 时刻时的真实值， S_t 为 t 加权平均后的值， β 为权重值。红线即是指数加权平均后的结果。

上图中 β 设为0.9，那么指数加权平均的计算结果为：

$$S_1 = Y_1$$

$$S_2 = 0.9S_1 + 0.1Y_2$$

...

$$S_{99} = 0.9S_{98} + 0.1Y_{99}$$

$$S_{100} = 0.9S_{99} + 0.1Y_{100}$$

...

那么第100天的结果就可以表示为：

$$S_{100} = 0.1Y_{100} + 0.1 * 0.9Y_{99} + 0.1 * (0.9)^2Y_{98} + \dots$$

动量梯度下降算法

动量梯度下降（Gradient Descent with Momentum）计算梯度的指数加权平均数，并利用该值来更新参数值。动量梯度下降法的整个过程为，其中 β 通常设置为0.9：

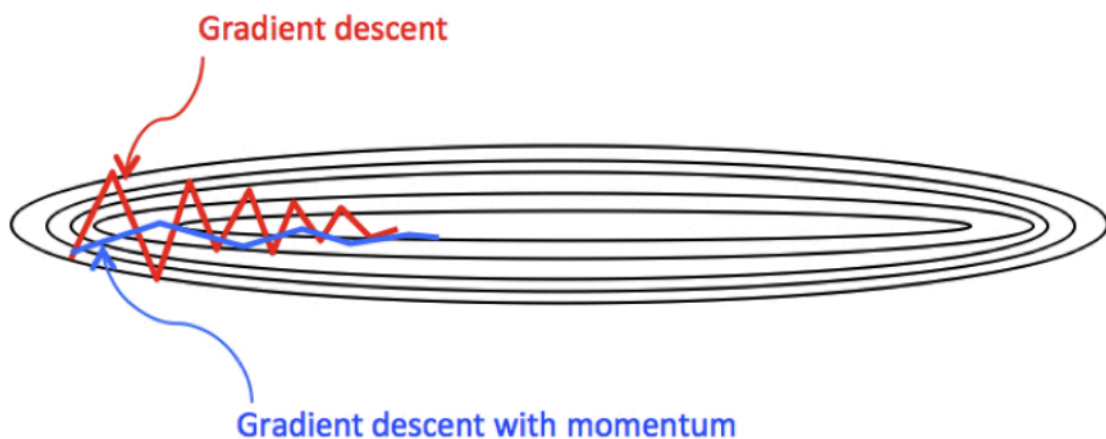
$$S_{dW}[l] = \beta S_{dW}[l] + (1 - \beta) dW[l]$$

$$S_{db}[l] = \beta S_{db}[l] + (1 - \beta) db[l]$$

$$W[l] := W[l] - \alpha S_{dW}[l]$$

$$b[l] := b[l] - \alpha S_{db}[l]$$

与原始的梯度下降算法相比，它的下降趋势更平滑。



在tf.keras中使用Momentum算法仍使用功能SGD方法，但要设置momentum参数，实现过程如下：

```
# 导入相应的工具包
import tensorflow as tf
# 实例化优化方法：SGD 指定参数beta=0.9
opt = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
# 定义要调整的参数，初始值
var = tf.Variable(1.0)
val0 = var.value()
# 定义损失函数
loss = lambda: (var ** 2)/2.0
#第一次更新：计算梯度，并对参数进行更新，步长为 `learning_rate * grad`
opt.minimize(loss, [var]).numpy()
val1 = var.value()
# 第二次更新：计算梯度，并对参数进行更新，因为加入了momentum,步长会增加
```

```

opt.minimize(loss, [var]).numpy()
val2 = var.value()
# 打印两次更新的步长
print("第一次更新步长={}".format((val0 - val1).numpy()))
print("第二次更新步长={}".format((val1 - val2).numpy()))

```

结果为：

```

第一次更新步长=0.10000002384185791
第二次更新步长=0.18000000715255737

```

另外还有一种动量算法Nesterov accelerated gradient(NAG)，使用了根据动量项**预先估计**的参数，在Momentum的基础上进一步加快收敛，提高响应性，该算法实现依然使用SGD方法，要设置nesterov设置为true.

3.2 AdaGrad

AdaGrad算法会使用一个小批量随机梯度 $\mathbf{g}_t$ 按元素平方的累加变量 $\mathbf{s}_t$ 。在首次迭代时，AdaGrad将 $\mathbf{s}_0$ 中每个元素初始化为0。在 t 次迭代，首先将小批量随机梯度 $\mathbf{g}_t$ 按元素平方后累加到变量 $\mathbf{s}_t$ ：

$$\mathbf{s}_t \leftarrow \mathbf{s}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$$

其中 $\odot$ 是按元素相乘。接着，我们将目标函数自变量中每个元素的学习率通过按元素运算重新调整一下：

$$\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \frac{\alpha}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t$$

其中 α 是学习率， ϵ 是为了维持数值稳定性而添加的常数，如

10^{-6} 。这里开方、除法和乘法的运算都是按元素运算的。这些按元素运算使得目标函数自变量中每个元素都分别拥有自己的学习率。

在tf.keras中的实现方法是：

```

tf.keras.optimizers.Adagrad(
    learning_rate=0.001, initial_accumulator_value=0.1, epsilon=1e-07
)

```

例子是：

```
# 导入相应的工具包
import tensorflow as tf
# 实例化优化方法: SGD
opt = tf.keras.optimizers.Adagrad(
    learning_rate=0.1, initial_accumulator_value=0.1, epsilon=1e-07
)
# 定义要调整的参数
var = tf.Variable(1.0)
# 定义损失函数: 无参但有返回值
def loss(): return (var ** 2)/2.0

# 计算梯度, 并对参数进行更新,
opt.minimize(loss, [var]).numpy()
# 展示参数更新结果
var.numpy()
```

3.3 RMSprop

AdaGrad算法在迭代后期由于学习率过小,能较难找到最优解。为了解决这一问题, RMSProp算法对AdaGrad算法做了一点小小的修改。

不同于AdaGrad算法里状态变量 s_t 是截至时间步 t 所有小批量随机梯度 g_t 按元素平方和, RMSProp (Root Mean Square Prop) 算法将这些梯度按元素平方做指数加权移动平均

$$s_{dw} = \beta s_{dw} + (1 - \beta)(dw)^2$$

$$s_{db} = \beta s_{db} + (1 - \beta)(db)^2$$

$$w := w - \frac{\alpha}{\sqrt{s_{dw} + \epsilon}} dw$$

$$b := b - \frac{\alpha}{\sqrt{s_{db} + \epsilon}} db$$

其中 ϵ 是一样为了维持数值稳定一个常数。最终自变量每个元素的学习率在迭代过程中就不再一直降低。RMSProp 有助于减少抵达最小值路径上的摆动, 并允许使用一个更大的学习率 α , 从而加快算法学习速度。

在tf.keras中实现时, 使用的方法是:


```
tf.keras.optimizers.RMSprop(  
    learning_rate=0.001, rho=0.9, momentum=0.0, epsilon=1e-07, centered=False,  
    name='RMSprop', **kwargs  
)
```

例子:

```
# 导入相应的工具包  
import tensorflow as tf  
# 实例化优化方法RMSprop  
opt = tf.keras.optimizers.RMSprop(learning_rate=0.1)  
# 定义要调整的参数  
var = tf.Variable(1.0)  
# 定义损失函数: 无参但有返回值  
def loss(): return (var ** 2)/2.0  
  
# 计算梯度, 并对参数进行更新,  
opt.minimize(loss, [var]).numpy()  
# 展示参数更新结果  
var.numpy()
```

输出结果为:

```
0.6837723
```

3.4 Adam

Adam 优化算法 (Adaptive Moment Estimation, 自适应矩估计) 将 Momentum 和 RMSProp 算法结合在一起。Adam算法在RMSProp算法基础上对小批量随机梯度也做了指数加权移动平均。

假设用每一个 mini-batch 计算 dW 、 db , 第 t 次迭代时:

$$vdW = \beta_1 vdW + (1 - \beta_1)dW$$

$$vdb = \beta_1 vdb + (1 - \beta_1)db$$

$$v_{dW}^{corrected} = \frac{v_{dW}^{[l]}}{1 - (\beta_1)^t}$$

$$sdW = \beta_2 sdW + (1 - \beta_2)(dW)^2$$

$$sdb = \beta_2 sdb + (1 - \beta_2)(db)^2$$

$$s_{dW}^{corrected} = \frac{s_{dW}^{[l]}}{1 - (\beta_2)^t}$$

其中l为某一层，t为移动平均第次的值

Adam 算法的参数更新：

$$W := W - \alpha \frac{v_{dW}^{corrected}}{\sqrt{s_{dW}^{corrected} + \epsilon}}$$

$$b := b - \alpha \frac{v_{db}^{corrected}}{\sqrt{s_{db}^{corrected} + \epsilon}}$$

建议的参数设置的值：

- 学习率 α ：需要尝试一系列的值，来寻找比较合适的
- β_1 ：常用的缺省值为 0.9
- β_2 ：建议为 0.999
- ϵ ：默认值1e-8

在tf.keras中实现的方法是：

```
tf.keras.optimizers.Adam(  
    learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07  
)
```

例子:

```
# 导入相应的工具包  
import tensorflow as tf  
# 实例化优化方法Adam  
opt = tf.keras.optimizers.Adam(learning_rate=0.1)  
# 定义要调整的参数  
var = tf.Variable(1.0)  
# 定义损失函数: 无参但有返回值  
def loss(): return (var ** 2)/2.0  
  
# 计算梯度, 并对参数进行更新,  
opt.minimize(loss, [var]).numpy()  
# 展示参数更新结果  
var.numpy()
```

结果为:

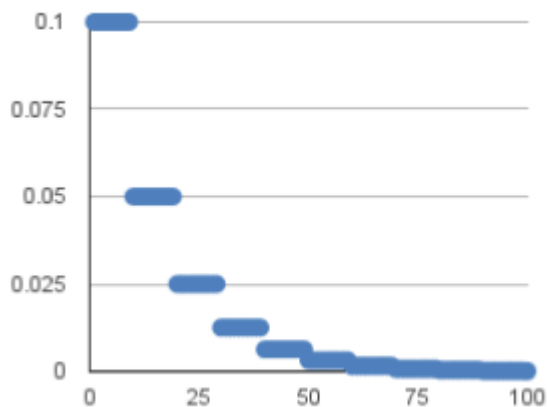
```
0.90000033
```

4.学习率退火

在训练神经网络时, 一般情况下学习率都会随着训练而变化, 这主要是由于, 在神经网络训练的后半期, 如果学习率过高, 会造成loss的振荡, 但是如果学习率减小的过快, 又会造成收敛变慢的情况。

4.1 分段常数衰减

分段常数衰减是在事先定义好的训练次数区间上, 设置不同的学习率常数。刚开始学习率大一些, 之后越来越小, 区间的设置需要根据样本量调整, 一般样本量越大区间间隔应该越小。



在tf.keras中对应的方法是：

```
tf.keras.optimizers.schedules.PiecewiseConstantDecay(boundaries, values)
```

参数：

- Boundaries: 设置分段更新的step值
- Values: 针对不用分段的学习率值

例子：对于前100000步，学习率为1.0，对于接下来的100000-110000步，学习率为0.5，之后的步骤学习率为0.1

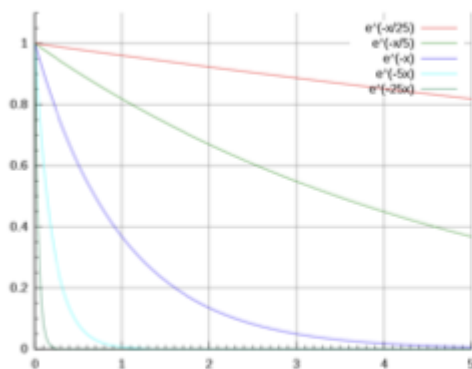
```
# 设置的分段的step值
boundaries = [100000, 110000]
# 不同的step对应的学习率
values = [1.0, 0.5, 0.1]
# 实例化进行学习的更新
learning_rate_fn = keras.optimizers.schedules.PiecewiseConstantDecay(
    boundaries, values)
```

4.2 指数衰减

指数衰减可以用如下的数学公式表示,

$$\alpha = \alpha_0 e^{-kt}$$

其中，t表示迭代次数， α_0 ,k是超参数



This plot shows decay for decay rates

在tf.keras中的实现是：

```
tf.keras.optimizers.schedules.ExponentialDecay(initial_learning_rate,
    decay_steps,
```

```
decay_rate)
```

具体的实现是：

```
def decayed_learning_rate(step):  
    return initial_learning_rate * decay_rate ^ (step / decay_steps)
```

参数：

Initial_learning_rate: 初始学习率, α_0

decay_steps: k值

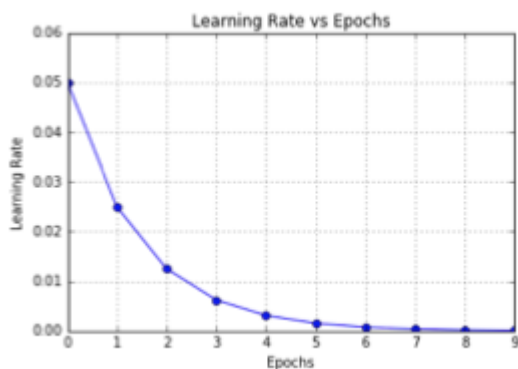
decay_rate: 指数的底

4.3 1/t衰减

1/t衰减可以用如下的数学公式表示：

$$\alpha = \alpha_0 / (1 + kt)$$

其中, t表示迭代次数, α_0, k 是超参数



在tf.keras中的实现是：

```
tf.keras.optimizers.schedules.InverseTimeDecay(initial_learning_rate,  
decay_steps,  
decay_rate)
```

具体的实现是：

```
def decayed_learning_rate(step):  
    return initial_learning_rate / (1 + decay_rate * step / decay_step)
```

参数:

Initial_learning_rate: 初始学习率, α_0

decay_step/decay_steps: k值

总结

- 知道梯度下降算法

一种寻找使损失函数最小化的方法: 批量梯度下降, 随机梯度下降, 小批量梯度下降

- 理解神经网络的链式法则

复合函数的求导

- 掌握反向传播算法 (BP算法)

神经网络进行参数更新的方法

- 知道梯度下降算法的优化方法

动量算法, adaGrad, RMSProp, Adam

- 了解学习率退火

分段常数衰减, 指数衰减, $1/t$ 衰减